



# Retrieval Augmented Generation for Knowledge-Intensive NLP tasks

**JIWON LEE**

Lewis, Patrick, et al. "Retrieval-augmented generation for knowledge-intensive nlp tasks." Advances in neural information processing systems 33 (2020): 9459-9474.

- **Pre-trained LLM (e.g., GPT-2)**

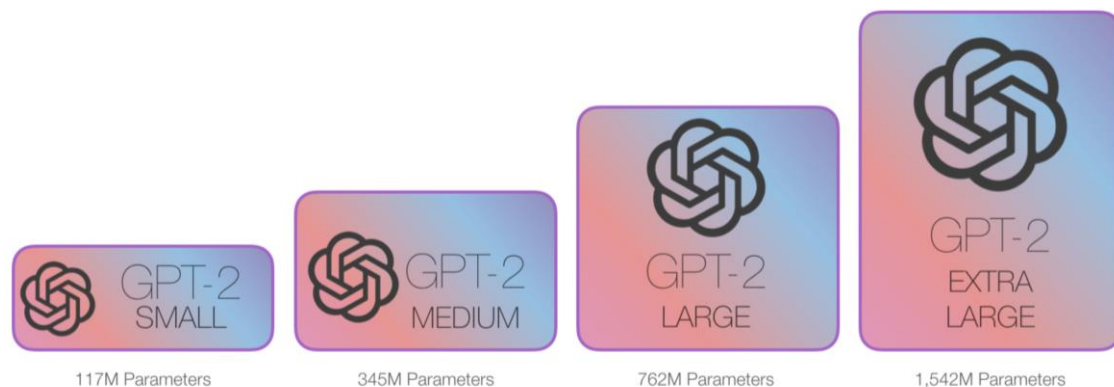
지식을 파라미터에 담음

-> 업데이트, 수정이 어려움

-> 근거 제시 어려움

-> hallucination 위험

**=> Parametric memory + Non-Parametric memory 결합**



Radford, Alec, et al. "Language models are unsupervised multitask learners." OpenAI blog 1.8 (2019): 9.

But no matter how compelling many of GPT-2 examples seem, the reality is that its representations are thin and unreliable, akin in to what Nie et al (2019) note above, often falling apart under close inspection (Marcus, 2020). Here are two typical cases, drawn from an in-development benchmark I presented at NeurIPS in December 2019 (Marcus, 2019).

- *Yesterday I dropped my clothes off at the dry cleaners and have yet to pick them up. Where are my clothes? at my mom's house.*
- *There are six frogs on a log. Two leave, but three join. The number of frogs on the log is now seventeen*

Marcus, Gary. "The next decade in AI: four steps towards robust artificial intelligence." arXiv preprint arXiv:2002.06177 (2020).

- **REALM** (Retrieval-Augmented Masked LM Pretraining)
- **Retriever로 문서  $z$ 를 가져와 [MASK] 토큰 예측에 활용**
- **단점:**

Open-domain Extractive QA 중심 → 문서에 정답 문자열이 그대로 있어야 유리

→ 자유서술/추상적 답변 생성에 제약

여러 문서 정보를 합성해서 한 문장으로 만들기 어려움

→ 단서가 여러 문서에 흩어지면 불리

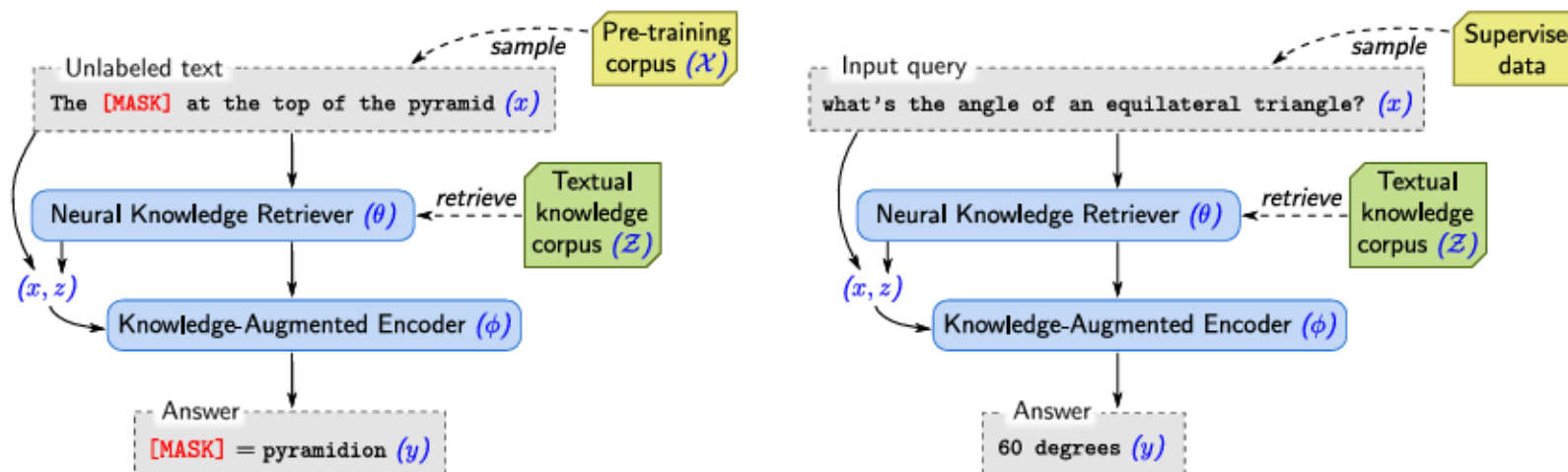
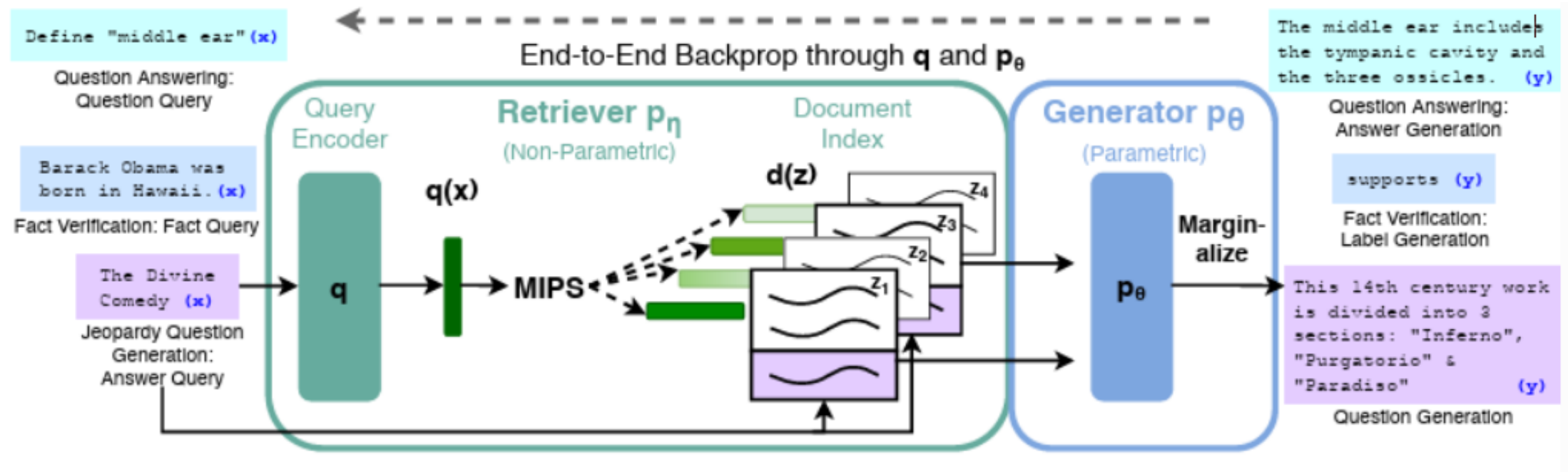


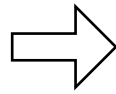
Figure 2. The overall framework of REALM. **Left:** *Unsupervised pre-training*. The knowledge retriever and knowledge-augmented encoder are jointly pre-trained on the unsupervised language modeling task. **Right:** *Supervised fine-tuning*. After the parameters of the retriever ( $\theta$ ) and encoder ( $\phi$ ) have been pre-trained, they are then fine-tuned on a task of primary interest, using supervised examples.

- Pretrained retriever (Query Encoder + Document Index)
- Pretrained seq2seq model
- Fine-tune end-to-end



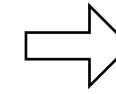
# Pretrained retriever: DPR (Dense Passage Retrieval)

Dense Encoder  $E_p$



[CLS] Title [SEP] Would you like a cup of coffee? [SEP]  
[CLS] Title [SEP] The weather is not good [SEP]  
[CLS] Title [SEP] Let's meet tomorrow [SEP]

21 million passages



BERT

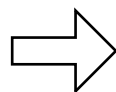


D = 768



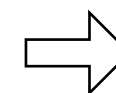
# Pretrained retriever: DPR (Dense Passage Retrieval)

## Dense Encoder $E_p$

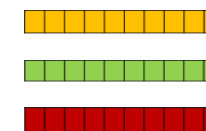


[CLS] Title [SEP] Would you like a cup of coffee? [SEP]  
[CLS] Title [SEP] The weather is not good [SEP]  
[CLS] Title [SEP] Let's meet tomorrow [SEP]

21 million passages



BERT

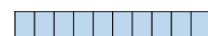
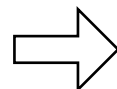


D = 768

## Query Encoder $E_q$

Who is the bad guy in  
lord of the rings ?

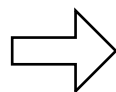
Question



D = 768

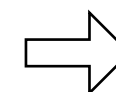
BERT

## Dense Encoder $E_p$

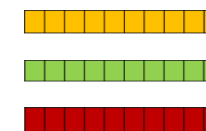


[CLS] Title [SEP] Would you like a cup of coffee? [SEP]  
[CLS] Title [SEP] The weather is not good [SEP]  
[CLS] Title [SEP] Let's meet tomorrow [SEP]

21 million passages



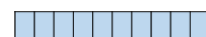
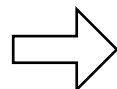
BERT



D = 768

## Query Encoder $E_q$

Who is the bad guy in  
lord of the rings ?



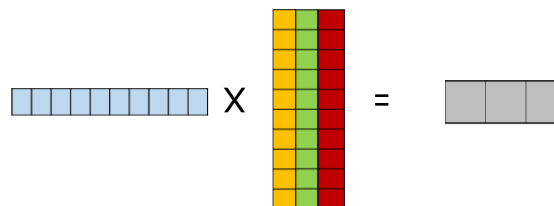
D = 768

Question

BERT

## Similarity (FAISS)

$$\text{sim}(q, p) = E_q(q)^T E_p(p)$$



$$(1, D) \cdot (D, B) = (1, B)$$

21 million  
passages 중  
해당 쿼리와  
유사도가 제일  
높은  
Top - k  
passage 추출

$$\begin{aligned} P_{\text{start},i}(s) &= \text{softmax}(\mathbf{P}_i \mathbf{w}_{\text{start}})_s, \\ P_{\text{end},i}(t) &= \text{softmax}(\mathbf{P}_i \mathbf{w}_{\text{end}})_t, \\ P_{\text{selected}}(i) &= \text{softmax}(\hat{\mathbf{P}}^T \mathbf{w}_{\text{selected}})_i, \end{aligned}$$

- **Bi-encoder architecture:** document encoder + query encoder

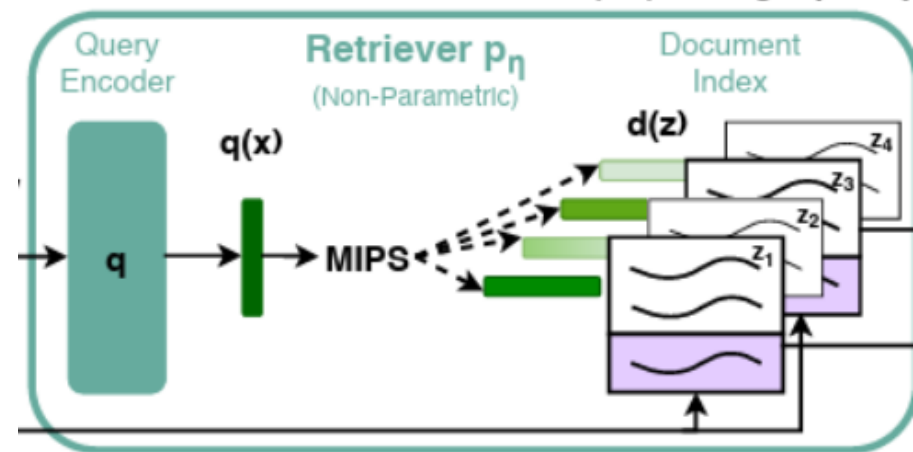
- **Document embedding**

$$d(z) = BERT_d(z)$$

Non-parametric memory = Document index  
Wikipedia(2018-12) → 100-word chunks → 21M passages

- **Query embedding**

$$q(x) = BERT_q(x)$$

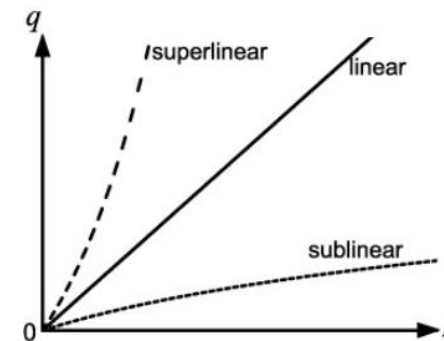


- **Retrieval score/ distribution**

$$p_{\eta}(z|x) \propto \exp(d(z)^{\top} q(x)), \quad z \in \text{top}K$$

Maximum Inner Product Search 를 수행해서 top-k 개의 문서를 선택  
sub-linear time

$BERT_d$  : freeze / document index : fixed /  $BERT_q$  : fine-tune





- **Generator distribution**

$$p_{\theta}(y_i | x, z, y_{1:i-1})$$

입력  $x$ , 검색 문서  $z$ , 이전 생성 토큰  $y_{<i}$  를 조건으로  
다음 토큰 생성

- **Parametric memory**

모델 파라미터  $\theta$ 에 저장된 언어 지식 + 배경 지식

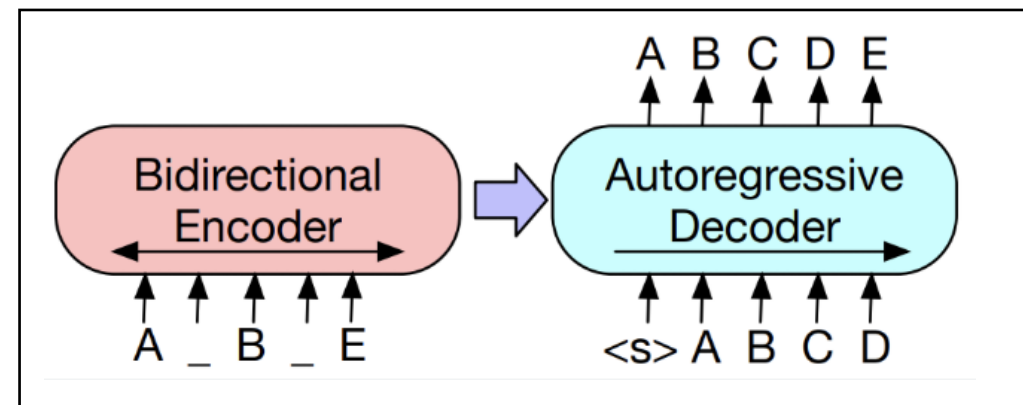
- **Backbone (encoder - decoder model)**

BART-large : pretrained seq2seq

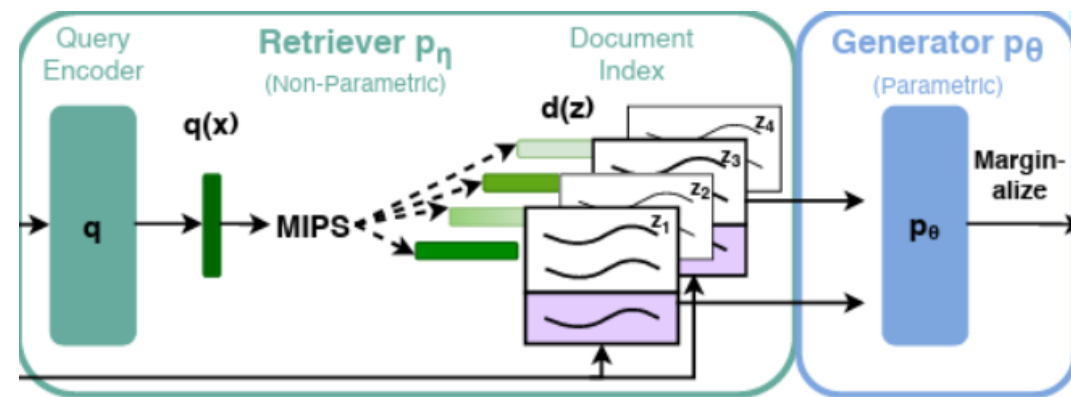
- **Input to BART:**

Encoder:  $[x, z]$  (query+ retrieved passage)

Decoder:  $y_{<i}$



Lewis, Mike, et al. "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension." *Proceedings of the 58th annual meeting of the association for computational linguistics*. 2020.



## • RAG-Sequence Model

하나의 문서  $z$  가 전체 출력 시퀀스  $y$  를 책임진다고 가정

$$p_{RAG-Sequence}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

문서마다 ‘완성 답변’을 만든 뒤, 문서 가중치까지 반영해 최종 답을 선택/평가

## • RAG-Token Model

토큰  $y_i$  마다 다른 문서  $z_i$  를 사용할 수 있다고 가정

각 토큰 단계에서 문서 mixture로 다음 토큰 분포를 구성

$$p_{RAG\_Token} \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

매 토큰마다 문서들을 섞어가며 생성 → 여러 문서 정보 합성에 유리

- **RAG-Sequence Model**

하나의 문서  $z$  가 전체 출력 시퀀스  $y$  를 책임진다고 가정

문서마다 ‘완성 답변’을 만든 뒤, 문서 가중치까지 반영해 최종 답을 선택/평가

$$p_{RAG-Sequence}(y|x) \approx \sum_{z \in top-k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) = \sum_{z \in top-k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

- **RAG-Sequence Model**

하나의 문서  $z$  가 전체 출력 시퀀스  $y$  를 책임진다고 가정

문서마다 '완성 답변'을 만든 뒤, 문서 가중치까지 반영해 최종 답을 선택/평가

$$p_{RAG-Sequence}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} \boxed{p_{\eta}(z|x) p_{\theta}(y|x, z)} = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

Input  $x$  일 때, 문서  $Z$  를 참고할 확률  
( $\eta$ : retriever parameter)

top -k 개의 각 확률을 더함

Input  $x$  와 그 문서  $Z$  일 때, sequence  $y$  를 만들 확률  
( $\theta$ : BART parameter)

- **RAG-Sequence Model**

하나의 문서  $z$  가 전체 출력 시퀀스  $y$  를 책임진다고 가정

문서마다 '완성 답변'을 만든 뒤, 문서 가중치까지 반영해 최종 답을 선택/평가

$$p_{RAG-Sequence}(y|x) \approx \sum_{z \in top-k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) = \sum_{z \in top-k(p(\cdot|x))} \boxed{p_{\eta}(z|x)} \prod_i^N \boxed{p_{\theta}(y_i|x, z, y_{1:i-1})}$$

Z 가 나올 확률 고정

seq2seq 확률 계산:  
다음 토큰 예측 확률

- **RAG-Sequence Model**

하나의 문서  $z$  가 전체 출력 시퀀스  $y$  를 책임진다고 가정

문서마다 ‘완성 답변’을 만든 뒤, 문서 가중치까지 반영해 최종 답을 선택/평가

$$p_{RAG-Sequence}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

질문  $x$ : 에펠탑은 어디 있어?

정답  $y$ : 파리에 있다.

Retriever:  $z_1, z_2, z_3$  을 가져옴

$p(z_1|x) = 0.6$  (파리/에펠탑 문서)

$p(z_2|x) = 0.3$  (프랑스 랜드마크 문서)

$p(z_3|x) = 0.1$  (엉뚱한 문서)

$p(\text{파리에 있다} \cdot |x, z_1) = 0.9$

$p(\text{파리에 있다} \cdot |x, z_2) = 0.5$

$p(\text{파리에 있다} \cdot |x, z_3) = 0.05$

$$p(y|x) = 0.6 \times 0.9 + 0.3 \times 0.5 + 0.1 \times 0.05$$



- **RAG-Token Model**

토큰  $y_i$ 마다 다른 문서  $z_i$ 를 사용

각 토큰 단계에서 문서 mixture로 다음 토큰 분포를 구성

매 토큰마다 문서들을 섞어가며 생성 → 여러 문서 정보 합성에 유리

$$p_{RAG\_Token} \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

- **RAG-Token Model**

토큰  $y_i$ 마다 다른 문서  $z_i$ 를 사용

각 토큰 단계에서 문서 mixture로 다음 토큰 분포를 구성

매 토큰마다 문서들을 섞어가며 생성 → 여러 문서 정보 합성에 유리

$$p_{RAG\_Token} \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

질문  $x$ : 에펠탑은 어디 있어?  
정답  $y$ : [파리에, 있다, .]  
Retriever:  $z_1, z_2$ 을 가져옴

$p(z_1|x) = 0.7$  (파리/에펠탑 문서)  
 $p(z_2|x) = 0.3$  (런던 관련 문서)

$$p_{RAG\_Token} \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

질문 x: 에펠탑은 어디 있어?  
정답 y: [파리에, 있다]  
Retriever:  $z_1, z_2$ 을 가져옴

$p(z_1|x) = 0.7$  (파리/에펠탑 문서)  
 $p(z_2|x) = 0.3$  (런던 관련 문서)

1. 첫번째 토큰 “파리에” 예측

$Z_1$ :  $p(\text{파리에}|x, z_1) = 0.8$

$Z_2$ :  $p(\text{파리에}|x, z_2) = 0.2$

$p(\text{파리에}|x) = 0.7 \times 0.8 + 0.3 \times 0.2 = 0.56 + 0.06 = 0.62$

$$p_{RAG\_Token} \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

질문 x: 에펠탑은 어디 있어?  
정답 y: [파리에, 있다]  
Retriever:  $z_1, z_2$ 을 가져옴

$p(z_1|x) = 0.7$  (파리/에펠탑 문서)  
 $p(z_2|x) = 0.3$  (런던 관련 문서)

1. 첫번째 토큰 “파리에” 예측

$Z_1$ :  $p(\text{파리에}|x, z_1) = 0.8$

$Z_2$ :  $p(\text{파리에}|x, z_2) = 0.2$

$$p(\text{파리에}|x) = 0.7 \times 0.8 + 0.3 \times 0.2 = 0.56 + 0.06 = 0.62$$

2. 두번째 토큰 “있다” 예측

$Z_1$ :  $p(\text{있다}|x, z_1, \text{파리에}) = 0.9$

$Z_2$ :  $p(\text{있다}|x, z_2, \text{파리에}) = 0.4$

$$p(\text{있다}|x) = 0.7 \times 0.9 + 0.3 \times 0.4 = 0.63 + 0.12 = 0.75$$

$$p_{RAG\_Token} \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

1. 첫번째 토큰 “파리에” 예측

$$p(\text{파리에}|x) = 0.7 \times 0.8 + 0.3 \times 0.2 = 0.56 + 0.06 = 0.62$$

2. 두번째 토큰 “있다” 예측

$$p(\text{있다}|x) = 0.7 \times 0.9 + 0.3 \times 0.4 = 0.63 + 0.12 = 0.75$$

$$p_{RAG\_Token} = 0.62 \times 0.75 = 0.465$$

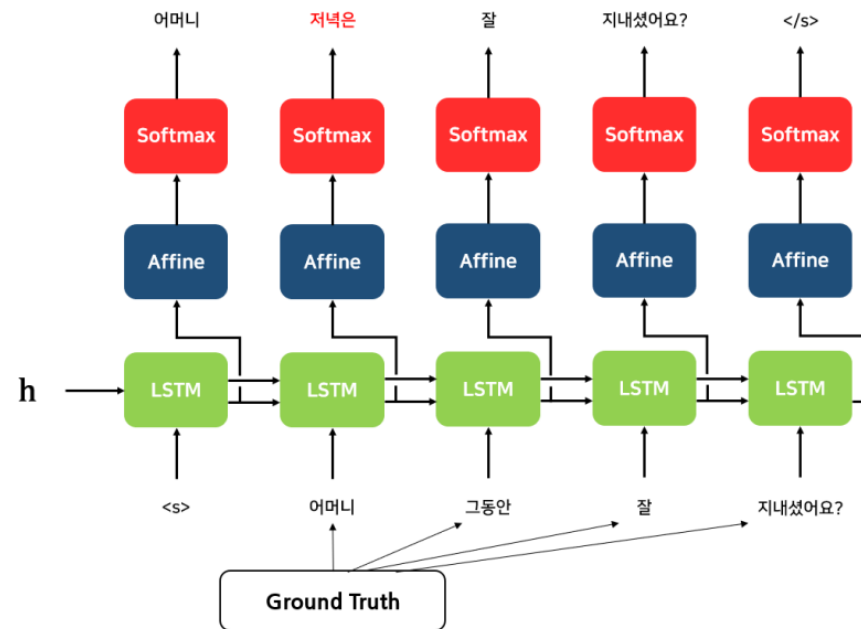
RAG-Token 장점: 여러 문서를 토큰 단위로 섞어서 더 유연하게 사실을 조합 가능

RAG-Token 단점: 문서가 토큰마다 바뀌는 효과 때문에 답이 덜 일관적이거나, 계산이 더 비쌀 수 있음

- **Fine tuning corpus:** 입력/출력 텍스트 쌍  $(x_j, y_j)$   
각 스텝에서 이전 토큰  $y_{1:i-1}$  는 모델 출력이 아닌 정답을 사용
- **Retrieval supervision 없음**  
정답 문서 라벨 없이 end-to-end로 학습  
 $p(y_j|x_j)$  가  $p_{\eta}(z|x)$  를 포함  $\rightarrow$  retriever 도 함께 업데이트 됨
- **Marginal negative Log likelihood**  
정답 토큰  $y_j$  를 더 잘 맞추도록 학습함.

$$L = \sum_j -\log p(y_j|x_j)$$

- 업데이트: Backpropagation (Adam)
- Fine tuning 대상 :  $BERT_q$  , BART



- **Decoding:**

정답이 없으므로 조건  $y_{1:i-1}$  는 지금까지 생성된 후보

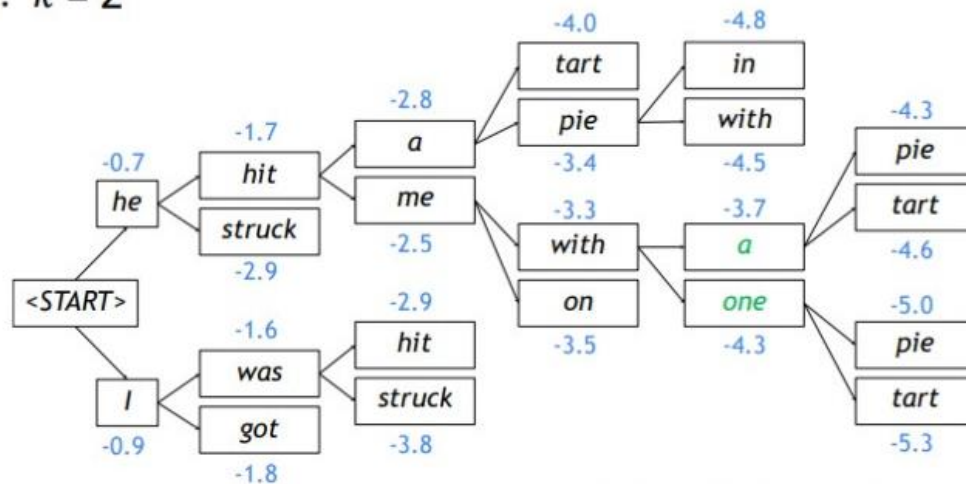
목표:  $y^* = \operatorname{argmax} p(y|x)$

전체 탐색이 불가능 -> beam search 로 근사

- Beam search:

각 step에서 탐색의 영역을 k개의 가장 가능도가 높은 토큰들로 유지하며 다음 단계를 탐색

size:  $k = 2$





# Decoding – RAG-Token

$$p_{RAG\_Token} \approx \prod_i \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

$$p'_{\theta}(y_i|x, y_{1:i-1}) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

모든 문서의 확률을 다 합쳐서, 다음 토큰 확률 정의  
일반적인 seq2seq 문장 생성과 같다.

$$p_{RAG-Token}(y|x) = \prod_i p'_{\theta}(y_i|x, y_{1:i-1})$$

=> Standard beam search

$$p_{\eta}(Z_1|x) = 0.6$$

|       |     |      |                                                                                     |
|-------|-----|------|-------------------------------------------------------------------------------------|
| $Z_1$ | 파리에 | 0.5  | $p_{\eta}(Z_1 x)p_{\theta}(\text{파리에}  x, Z_1, y_{1:i-1}) = 0.6 \times 0.5 = 0.3$   |
|       | 런던에 | 0.3  | $p_{\eta}(Z_1 x)p_{\theta}(\text{런던에}  x, Z_1, y_{1:i-1}) = 0.6 \times 0.3 = 0.18$  |
|       | 미국에 | 0.15 | $p_{\eta}(Z_1 x)p_{\theta}(\text{미국에}  x, Z_1, y_{1:i-1}) = 0.6 \times 0.15 = 0.09$ |
|       | 한국에 | 0.05 | $p_{\eta}(Z_1 x)p_{\theta}(\text{한국에}  x, Z_1, y_{1:i-1}) = 0.6 \times 0.05 = 0.03$ |

$$p_{\eta}(Z_2|x) = 0.4$$

|       |     |      |                                                                                      |
|-------|-----|------|--------------------------------------------------------------------------------------|
| $Z_2$ | 파리에 | 0.45 | $p_{\eta}(Z_2 x)p_{\theta}(\text{파리에}  x, Z_2, y_{1:i-1}) = 0.4 \times 0.45 = 0.18$  |
|       | 런던에 | 0.35 | $p_{\eta}(Z_2 x)p_{\theta}(\text{런던에}  x, Z_2, y_{1:i-1}) = 0.4 \times 0.35 = 0.14$  |
|       | 미국에 | 0.01 | $p_{\eta}(Z_2 x)p_{\theta}(\text{미국에}  x, Z_2, y_{1:i-1}) = 0.4 \times 0.01 = 0.004$ |
|       | 한국에 | 0.19 | $p_{\eta}(Z_2 x)p_{\theta}(\text{한국에}  x, Z_2, y_{1:i-1}) = 0.4 \times 0.19 = 0.076$ |

$$\text{파리에} = 0.3 + 0.18 = 0.48$$

$$\text{런던에} = 0.18 + 0.14 = 0.32$$

$$\text{미국에} = 0.09 + 0.04 = 0.094$$

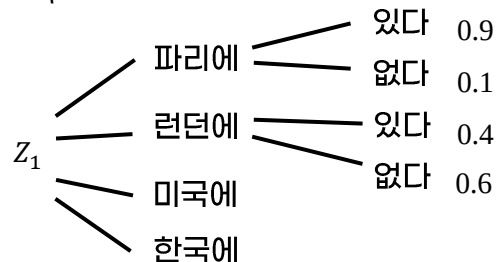
$$\text{한국에} = 0.03 + 0.076 = 0.106$$

**Beam = 2**

파리에

런던에

$$p_{\eta}(Z_1|x) = 0.6$$



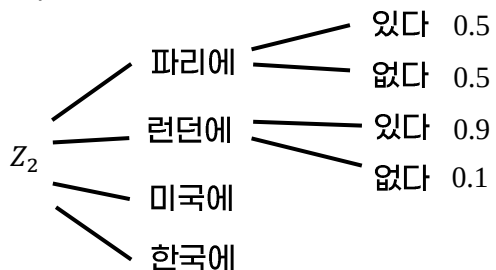
$$p_{\eta}(Z_1|x)p_{\theta}(\text{있다} | x, Z_1, \text{파리에}) = 0.6 \times 0.9 = 0.54$$

$$p_{\eta}(Z_1|x)p_{\theta}(\text{없다} | x, Z_1, \text{파리에}) = 0.6 \times 0.1 = 0.06$$

$$p_{\eta}(Z_1|x)p_{\theta}(\text{있다} | x, Z_1, \text{런던에}) = 0.6 \times 0.4 = 0.24$$

$$p_{\eta}(Z_1|x)p_{\theta}(\text{없다} | x, Z_1, \text{런던에}) = 0.6 \times 0.6 = 0.36$$

$$p_{\eta}(Z_2|x) = 0.4$$



$$p_{\eta}(Z_2|x)p_{\theta}(\text{있다} | x, Z_2, \text{파리에}) = 0.4 \times 0.5 = 0.2$$

$$p_{\eta}(Z_2|x)p_{\theta}(\text{없다} | x, Z_2, \text{파리에}) = 0.4 \times 0.5 = 0.2$$

$$p_{\eta}(Z_2|x)p_{\theta}(\text{있다} | x, Z_2, \text{런던에}) = 0.4 \times 0.9 = 0.36$$

$$p_{\eta}(Z_2|x)p_{\theta}(\text{없다} | x, Z_2, \text{런던에}) = 0.4 \times 0.1 = 0.04$$

$$\text{파리에 있다} = 0.54 + 0.2 = 0.74$$

$$\text{파리에 없다} = 0.06 + 0.2 = 0.26$$

$$\text{런던에 있다} = 0.24 + 0.36 = 0.6$$

$$\text{런던에 없다} = 0.36 + 0.04 = 0.4$$

$$p_{RAG\_Token} \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

1번째 토큰 확률까지 곱함

$$\text{파리에 있다} = 0.48 \times 0.74 = 0.3552$$

$$\text{파리에 없다} = 0.48 \times 0.26 = 0.1248$$

$$\text{런던에 있다} = 0.32 \times 0.6 = 0.192$$

$$\text{런던에 없다} = 0.32 \times 0.40 = 0.128$$

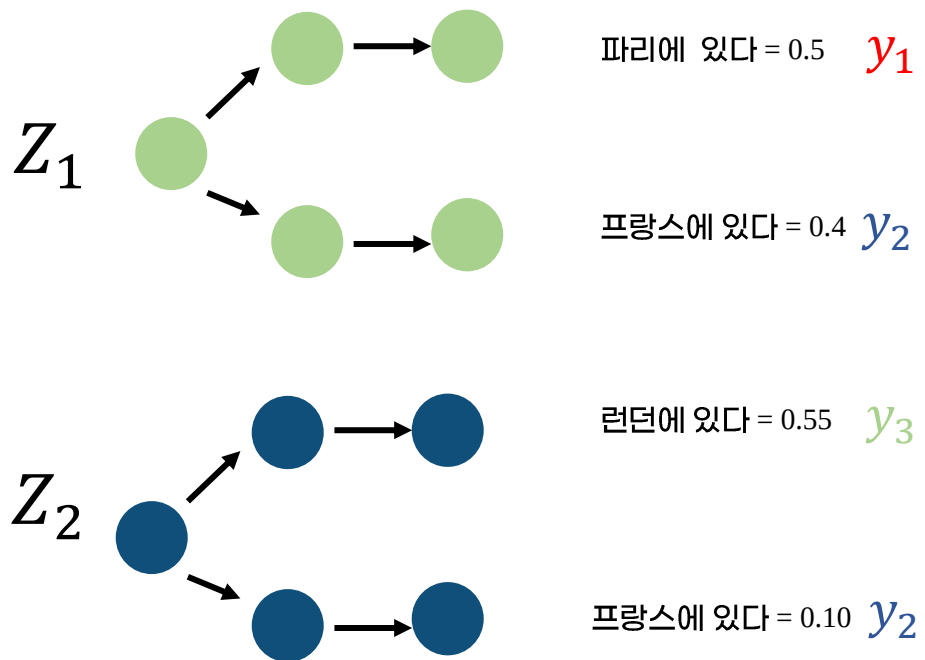
**Beam = 2**

파리에 있다

런던에 있다

$$p_{RAG-Sequence}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

RAG-sequence :



$$p(y_1|x) = p_{\eta}(Z_1|x)p_{\theta}(y_1|x, Z_1) + p_{\eta}(Z_2|x)p_{\theta}(y_1|x, Z_2) \quad ?$$

$$p(y_2|x) = p_{\eta}(Z_1|x)p_{\theta}(y_2|x, Z_1) + p_{\eta}(Z_2|x)p_{\theta}(y_2|x, Z_2)$$

$$p(y_3|x) = p_{\eta}(Z_1|x)p_{\theta}(y_3|x, Z_1) + p_{\eta}(Z_2|x)p_{\theta}(y_3|x, Z_2) \quad ?$$

문제 상황:

어떤 후보는 어떤 문서 빔에 등장하지 않을 수 있다.

1. Thorough Decoding  
출력 문장을 강제로  $y$  로 놓은 채, 확률을 계산한다.
2. Fast Decoding  
문서의 기여도를 0 으로 처리한다.

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix D for further details.

|             | Model          | NQ          | TQA               | WQ          | CT          |
|-------------|----------------|-------------|-------------------|-------------|-------------|
| Closed Book | T5-11B [52]    | 34.5        | - /50.1           | 37.4        | -           |
|             | T5-11B+SSM[52] | 36.6        | - /60.5           | 44.7        | -           |
| Open Book   | REALM [20]     | 40.4        | - / -             | 40.7        | 46.8        |
|             | DPR [26]       | 41.5        | <b>57.9</b> / -   | 41.1        | 50.6        |
|             | RAG-Token      | 44.1        | 55.2/66.1         | <b>45.5</b> | 50.0        |
|             | RAG-Seq.       | <b>44.5</b> | 56.8/ <b>68.0</b> | 45.2        | <b>52.2</b> |

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [4], FEVER-3 is [68] and FEVER-2 is [57] \*Uses gold context/evidence. Best model without gold access underlined.

| Model    | Jeopardy    |             | MSMARCO      |              | FVR3        | FVR2         |
|----------|-------------|-------------|--------------|--------------|-------------|--------------|
|          | B-1         | QB-1        | R-L          | B-1          | Label       | Acc.         |
| SotA     | -           | -           | <b>49.8*</b> | <b>49.9*</b> | <b>76.8</b> | <b>92.2*</b> |
| BART     | 15.1        | 19.7        | 38.2         | 41.6         | 64.0        | 81.1         |
| RAG-Tok. | <b>17.3</b> | <b>22.2</b> | 40.1         | 41.5         | 72.5        | <u>89.5</u>  |
| RAG-Seq. | 14.7        | 21.4        | <u>40.8</u>  | <u>44.2</u>  |             |              |

**Effect of Retrieving more documents** Models are trained with either 5 or 10 retrieved latent documents, and we do not observe significant differences in performance between them. We have the flexibility to adjust the number of retrieved documents at test time, which can affect performance and runtime. Figure 3 (left) shows that retrieving more documents at test time monotonically improves Open-domain QA results for RAG-Sequence, but performance peaks for RAG-Token at 10 retrieved documents. Figure 3 (right) shows that retrieving more documents leads to higher Rouge-L for RAG-Token at the expense of Bleu-1, but the effect is less pronounced for RAG-Sequence.

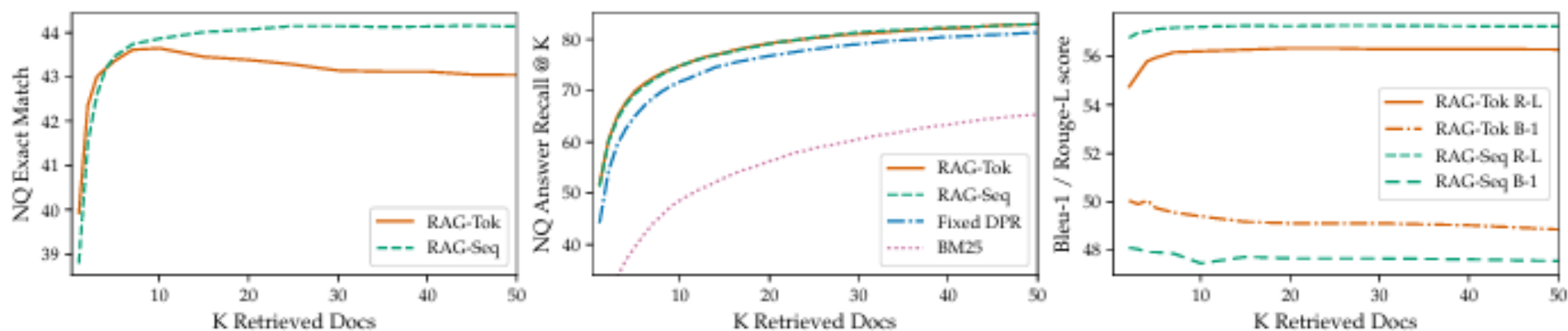


Figure 3: Left: NQ performance as more documents are retrieved. Center: Retrieval recall performance in NQ. Right: MS-MARCO Bleu-1 and Rouge-L as more documents are retrieved.